Recommended Tech Stack- Emotional Awareness Mobile Application

1) Mobile App

Flutter

Why this is recommended

- One codebase for both iOS and Android keeps development cost much lower than separate native apps.
- Very suitable for the kind of polished, animation-friendly mobile experience shown in the presentation and app mockups.
- Works well for:
    - real-time UI updates
    - chat screens
    - push notification flows
    - contact access
    - map/location features for later phase
- Faster release and maintenance cycle for an MVP and Phase 2.

2) Backend API

Node.js with NestJS

Why NestJS

- Circl needs structured handling of:
    - authentication
    - users and circles
    - emotion check-ins
    - messaging
    - notifications
    - scheduled jobs
    - future premium analytics
- NestJS gives better architecture than a loose Express codebase, while still keeping development cost reasonable.
- Easier to maintain as features expand from MVP to premium.

3) Real-Time Layer

Socket.IO on Node.js

Why

- Real-time updates are explicitly required for emotion sharing, messaging, and notifications.
- Socket.IO is easier and cheaper to implement for MVP than more complex event-driven or streaming infrastructure.
- Works well for:
    - instant emotion status updates
    - direct chat
    - group chat
    - live Circl presence/state refresh

4) Database

PostgreSQL

Why PostgreSQL

- Strong relational fit for:
    - users
    - circles
    - memberships
    - invites
    - emotions
    - privacy rules
    - chat threads/messages
    - audit logs
- Better long-term structure than NoSQL for this type of relationship-heavy app.
- Useful for future reporting and emotional trend analytics in premium phase.

5) Cache / Queue / Background Jobs

Redis

Why
Redis will help with:

- socket session scaling
- unread counters
- rate limiting
- temporary invite/session data
- background queue support

6) Authentication

JWT-based auth + Apple Sign-In + Google Sign-In

Recommended setup

- Social sign-in through Apple and Google, as required in MVP.
- Backend issues its own JWT access and refresh tokens after social login
- Optional phone-based invite verification later if needed

Why this approach

- Keeps authentication flexible
- Avoids locking the whole backend architecture into a BaaS platform
- Easier to extend for admin tools and compliance logging later

7) Push Notifications

Firebase Cloud Messaging (FCM) + APNs via Flutter integration, triggered from Node backend

Why

- Lowest-friction push setup for Flutter apps
- Good for:
    - reminder notifications
    - new message notifications
    - emotion request notifications
    - smart alerts

      These are all explicitly required.
- Cheaper and simpler for MVP than building around a heavier marketing notification platform

8) File / Media Storage

Amazon S3

Use for

- profile pictures
- future media attachments if messaging expands
- exported user data for GDPR requests
- audit/support artifacts if needed

Why

- Simple, standard, low-maintenance
- Easy to secure with private buckets and signed URLs

9) Hosting & Deployment

AWS